Rapport d'usage de l'IA — TP1

Modèle utilisé

Assistant : Claude (Anthropic), dans Claude.ai.
Modèle : Claude Sonnet 5 (vérifié dans le sélecteur de modèle de l'interface).
Je n'ai pas suivi précisément ma consommation de tokens au fil du TP ; la documentation Anthropic (docs.claude.com) explique où consulter cette information selon l'interface utilisée (claude.ai, API).

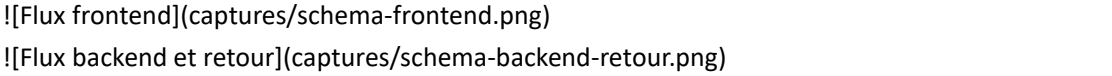
Usage de l'IA sur ce TP

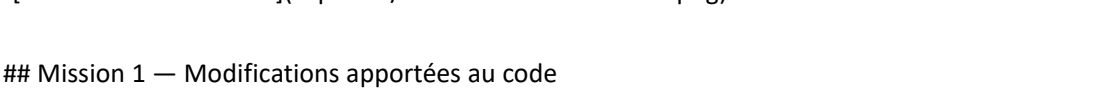
L'assistant a été utilisé pour m'aider à déboguer la configuration MongoDB Atlas, comprendre l'architecture du projet fourni, et écrire le code de la Mission 1 (déconnexion, gestion du 401, validations de formulaire, chargement automatique du profil). Chaque modification m'a été expliquée ligne par ligne pour que je puisse la reproduire et la défendre à l'oral.

Mission 0 — Cartographie de l'application

- Composant racine : `app/components/app/app.ts`
- Routes : `app/routes.ts` (`/login`, `/register` publiques ; `/profile`, `/tracks` protégées par `authGuard`)
- Enregistrement de HttpClient : `main.ts`, via `provideHttpClient(withInterceptors([authInterceptor]))`
- Ajout du JWT : `app/shared/interceptors/auth.interceptor.ts`

Schéma annoté du flux de connexion :





Mission 1 — Modifications apportées au code

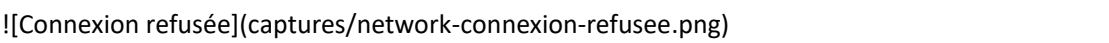
1. `app/components/app/app.ts` et `app.html` : bouton de déconnexion et navigation réactive selon `auth.currentUser()`.
2. `app/shared/interceptors/auth.interceptor.ts` : gestion du `401`, déconnexion et redirection automatique vers `/login`.
3. `register-page.ts` / `.html` et `login-page.ts` / `.html` : validations (email, longueur du mot de passe) et messages d'erreur par champ.
4. `profile-page.ts` : chargement automatique du profil via `ngOnInit()`.

Routes backend utilisées pendant le TP : `POST /api/auth/login`, `POST /api/auth/register`, `GET /api/users/me`, `PUT /api/users/me`, `GET /api/tracks`, `POST /api/tracks`.

La mise à jour du profil utilisateur s'effectue :
- côté front : `profile-page.ts` (méthode `save()`) → `AuthService.update()` dans `shared/services/auth.service.ts`, qui appelle `PUT /api/users/me`.
- côté back : la route correspondante dans `backend/src/app.js`, qui met à jour le document dans la collection `users` via Mongoose (`backend/src/models/User.js`).

Checkpoint Network

1. Connexion réussie
Validée dans Chrome DevTools : `POST /api/auth/login` → statut `200`.

2. Connexion refusée

`POST /api/auth/login` → statut `401`, réponse `{"message":"Identifiants incorrects"}`.

3. Lecture de /api/users/me

`GET /api/users/me` → statut `200 OK`.

Signal vs localStorage

- `Signal` (`currentUser`, `token`) : état réactif en mémoire, propre à l'instance de l'application dans l'onglet ouvert. Il pilote automatiquement l'affichage (navigation, profil) dès qu'il change, mais disparaît si la page est rechargée.
- `localStorage` : stockage persistant dans le navigateur, qui survit au rechargement de la page et à la fermeture de l'onglet. Le JWT y est sauvegardé pour que la connexion ne soit pas perdue à chaque F5, mais lui seul ne met pas l'interface à jour automatiquement : c'est le Signal, relu au démarrage du service, qui fait ce lien.

Où trouver les traces du backend

Dans le terminal où `npm start` a été lancé pour `backend/` : chaque requête y affiche une ligne `[http] MÉTHODE /chemin -> code (durée)`. Si ce terminal est fermé, les traces ne sont plus visibles tant que le backend n'est pas relancé.